

Using LatticeFold to Prove an FHE Bootstrap

Daniel Rubin

April 8, 2025

Abstract

Your abstract.

1 Introduction

2 The bootstrap operation

3 Encoding the bootstrap circuit in LatticeFold CCS

3.1 Modulus Switch

3.2 Blind Rotation

Recall that each step of the blind rotation is a circuit that takes as input an integer $0 \leq a < 2N$, a ciphertext $c \in R_q$, and a GGSW encryption C_s of a bit of the secret key s . Using the 64-bit Goldilocks prime q and a decomposition parameter $B = 2^{16}$, $C_s \in R_q^4$. The result of the circuit is a new ciphertext $\tilde{c} \in R_q$ that decrypts to the same plaintext as c if $s = 0$, or $X^a c$ (c rotated a steps) if $s = 1$. In TFHE, this is carried out by the CMUX operation

$$\tilde{c} = (X^a c - c) \odot C_s + c \quad (1)$$

where $\odot$ represents the external product. The external product of a GLWE ciphertext u and GGSW ciphertext v is the inner product of the 16-bit decomposition of u with v , itself a vector of encryptions of 4 16-bit shifts of a message.

We wish to encode this circuit into a ring-CCS instance. The equation of a satisfying ring-CCS instance is

$$\sum_{i=1}^{n_s} m_i \cdot \bigcirc_{j \in S_i} (M_j \cdot \vec{z}) = 0^{nr}. \quad (2)$$

Here is a naive approach to choose appropriate parameters of a ring-CCS instance encoding a step of the blind rotation:

First, a simple but inefficient way to encode the product of c and X^a . The difficulty of encoding this operation is that a is not known ahead of time and the map $a \mapsto X^a$ is not linear over R_q . The Zama team solved this by using a binary decomposition of a . We must use one ring-valued input slot for the integer a and then 11 ($2N = 2048 = 2^{11}$) additional slots, one each for each bit of a binary decomposition $\sum 2^i a_i = a$. We write

$$X^a = \prod_{i=0}^{10} (1 + a_i (X^{2^i} - 1)). \quad (3)$$

Set a witness entry w_0 to represent the product $X^a c$. To encode this product, we need 13 matrices. We need 11 matrices $(M_{prod})_i$ of the form

$$(M_{prod})_{i,rc} = \begin{cases} X^{2^i} - 1 & \text{if } r = 0, c \leftrightarrow a_i \\ 1 & \text{if } r = 0, c \leftrightarrow 1 \\ 0 & \text{otherwise} \end{cases}, \quad (4)$$

where the rc in the subscript stands for row and column (hopefully no confusion will arise between c for column and c being the column corresponding to the ciphertext witness). We need another matrix for the c factor

$$(M_{prod})_{11,rc} = \begin{cases} 1 & r = 0, c \leftrightarrow c \\ 0 & \text{otherwise} \end{cases} \quad (5)$$

and a final matrix for setting the product equal to the witness entry w_0 :

$$(M_{prod})_{12,rc} = \begin{cases} -1 & r = 0, c \leftrightarrow w_0 \\ 0 & \text{otherwise} \end{cases} \quad (6)$$

with the product enforced by choosing a first multiset $S_1 = \{0, 1, \dots, 11\}$, that is, a product of 12 terms, and the witness entry in that row is not included in the product. Thus row 0 encodes the product of X^a and c .

With this approach we additionally need to encode the 11 constraints

$$a_i(1 - a_i) = 0 \quad (7)$$

each of which takes up a row and two matrices, plus another matrix for the binary decomposition enforcing

$$\sum a_i 2^i = a. \quad (8)$$

Note: I suspect that a better way to encode the multiplication by X^a is by means of a lookup table. The table need only consist of 2048 entries, enforcing the map $a \mapsto X^a$. Possibly the vector of all 728 integer inputs for all of the steps of the bootstrap can be checked in the lookup table in a single instance. Even better would be if there were a way to encode the specific action of a rotation: Note that the multiplication in R_q by X^a is especially simple, amounting to a shift of the coefficient entries with sign changes, for $O(N)$ basic operations, whereas general multiplications of ring elements require $O(N \log N)$ *field* operations. Is there any way to take advantage of this?

The last step is to enforce the CMUX operation. The only complication is that we must create witnesses for the log B -bit decompositions of the witness entries c and $w_0 = X^a c$, that is,

$$\sum_{i=0}^3 B^i c_i = c, \quad (9)$$

with each $c_i \in R_q$, and similarly for w_0 . Note that this is compatible with the decomposition in LatticeFold, in which the input witnesses to Ajtai commitments must have bounded entries. If the decomposition parameter from TFHE is chosen to be equal to the decomposition parameter in LatticeFold, then we do not have to do an additional decomposition before commitment.

The next unoccupied row will encode the CMUX operation with 9 matrices: 1 for each of the linear factors $(w_0)_i - c_i$, 4 for the factors $(C_s)_i$, and then a final matrix with coefficients 1 and -1 for c and $\tilde{c}$ in that row, and with the multisets chosen so that $(w_0)_i - c_i$ is paired with $(C_s)_i$. All the coefficients m are 1.

Can we remove the entries for the full ring elements and work exclusively with the decompositions? This will require a slight change to the $X^a c = w_0$ constraints. If so, this gives 4 witness elements each for c , $X^a c$, $\tilde{c}$, and C_s . Plus there is the need to include the constant, the integer a and the 11 entries in the binary decomposition. Therefore we must pad the witness vector size to the next power of 2 which is 32, and even if we used a lookup to eliminate the binary decomposition of a , it does not seem possible to reduce the size to 16.

Question: If it is possible to use a lookup relation to enforce the map $a \mapsto X^a$, and therefore one can encode the product of c and X^a with only two terms and not 12, would it be more efficient to use an R1CS instance for a step of the blind rotation as opposed to CCS?

3.3 Key Switching and Sample Extraction

3.4 Extending LatticeFold to handle lookup relations

Question: How can we fold both lookup instances and R1CS/CCS instances?

The LatticeFold paper describes a folding scheme that can attest to arithmetic relations described by ring-valued CCS instances. It is left for future work to incorporate lookup arguments into the LatticeFold scheme. We describe this effort in the following.

A lookup argument is

4 Folding in a tree

One of the main advantages of employing a folding strategy to generate a SNARK is that the processing of commitments to chunks and accumulation can be made highly parallel by folding in a tree. Following the Mangrove paper, we propose to commit to each of the subcircuit relations above, together with the calculation of partial products enforcing copy constraints (leaf nodes), all in parallel. Then in subsequent layers, we pair the nodes off and fold together. Each layer of folding can be performed in parallel.

4.1 Enforcing copy constraints

In order for the folded commitment to attest to the entire bootstrap circuit, it is necessary to enforce the *copy constraints* in addition to the chunk/subcircuit relations described above. By copy constraints we mean the wiring or identity relations between the witness elements from distinct subcircuits, such as that

$$x_{1,out} = x_{2,in}.$$

Copy constraints can be realized as a permutation relation, an equality of sets of the form

$$\bigcup \{(i, x_i)\} = \bigcup \{(i, x_{\sigma(i)})\}. \quad (10)$$

One strategy to attest to this equality of sets is to compute a *grand product*

$$\prod_i \frac{H(i, x_i)}{H(i, x_{\sigma(i)})} \quad (11)$$

which must be 1 at the end of the circuit. Here H is a universal hash function

$$H(i, x) = x + i \cdot \alpha + \beta \quad (12)$$

with α, β being verifier challenges chosen after commitment to the witnesses.

Therefore what Mangrove appears to propose is to compute at each leaf, in addition to the commit-

ment for the CCS relation, the partial products
$$\frac{\prod_{i \in N_{leaf}} H(i, x_i)}{\prod_{i \in D_{leaf}} H(i, x_{\sigma(i)})} =$$
 where the products are over witness

indices i such that $\sigma(i)$ is the witness of an index outside the chunk in that leaf. Then at interior nodes,

the partial products from child nodes are given as input, and the node computes
$$\frac{N_{node}}{D_{node}} = \frac{N_{child1} \cdot N_{child2}}{D_{child1} \cdot D_{child2}}$$

Issue 1: What is the appropriate analogue of H in our setting, where the witness elements are ring-valued? Must H, α, β all be ring-valued, or can H at least take values in a field? Must check original proof of soundness (in Plonk paper). Or must the ring-valued witnesses be split into their components?

Issue 2: Can we get away with only computing partial product terms that correspond to witnesses used across chunks, and not the terms for witnesses entirely local to a particular chunk? This seems safe because CCS encodes copy constraints within a chunk.

Issue 3: Is the partial product/grand product strategy best? Is there a logarithmic derivative-based additive version of this check? Would this be more efficient?

Issue 4: What is the role of the Merkle commitments at each leaf (node?) in Mangrove? Do Ajtai commitments take the place of these?